ShieldScrape: Secure & Scalable Cloud-Based Web Data Collection

Aryan Mayur Dhuru
Computer Science
Indiana University Bloomington
Bloomington, Indiana, USA
adhuru@iu.edu

Suhana Ambol
Data Science
Indiana University Bloomington
Bloomington, Indiana, USA
sambol@iu.edu

Varunkumar Sonawane
Computer Science
Indiana University Bloomington
Bloomington, Indiana, USA
vsonawa@iu.edu

Shivali Mate
Computer Science
Indiana University Bloomington
Bloomington, Indiana, USA
shimate@iu.edu

***Abstract*—This paper presents ShieldScraper, an automated, secure, and scalable cloud-based data engineering pipeline designed for advanced web scraping, data transformation, storage, and visualization. Implemented using a robust suite of Amazon Web Services (AWS), ShieldScraper integrates ECS Fargate for container orchestration, Glue for Extract-Transform-Load (ETL) processes, DynamoDB for scalable data storage, Athena for querying capabilities, and QuickSight for intuitive data visualization. The pipeline employs real-time monitoring and automated alerting, ensuring system robustness, fault tolerance, and operational efficiency.**

TABLE I. CLOUD COMPUTING TECHNIQUES

ID	**Component**	**Technologies used**
1	Scraping & Containerization	Scrapy, Docker, AWS ECR
2	Orchestration & Automation	AWS EventBridge, AWS ECS Fargate
3	Storage & Data Handling	AWS S3, AWS DynamoDB, AWS Glue
4	ETL & Analytics	AWS Glue ETL, AWS Glue Crawler, AWS Athena
5	Visualization & Monitoring	AWS QuickSight, AWS CloudWatch, AWS SNS

TABLE II. HIGHLIGHTS OF THE PROJECT

ID	**Highlight**
1	Fully automated orchestration and execution using AWS EventBridge and ECS Fargate
2	Secure data handling and scalable storage via AWS DynamoDB and S3
3	Real-time monitoring and immediate alerting mechanisms integrated with AWS SNS
4	Comprehensive data analytics and visualization through AWS QuickSight dashboards

I. INTRODUCTION

Shield Scrape aims to solve the challenge of manually tracking football league standings by automating the process of extracting and updating current standings from top leagues. With match data frequently changing, fans and analysts lack a consistent, real-time source for league positions. Shield Scrape addresses this by building an automated, cloud-native pipeline that scrapes, processes, and stores live standings every match day, delivering structured, up-to-date data across multiple leagues.

Shield Scrape will be resilient to web structure changes, compliant with scraping limitations, and scalable to support additional leagues and competitions. By integrating AWS services such as S3, Glue, DynamoDB, Lambda, and Fargate, Athena, Quicksite we aim to build a maintainable and extensible solution that supports downstream visualization, analytics, and alerting systems.

II. MOTIVATION

The primary goal of Shield Scrape is to create an automated system that reliably scrapes and processes real-time football standings from the top leagues around the world, including the English Premier League, La Liga, Serie A, and Bundesliga. These standings are to be extracted with high accuracy, structured consistently, and stored securely in a cloud environment for further processing and analysis. The project aims to reduce the dependency on manual scoreboard tracking and enable seamless integration with analytical tools and dashboards.

To achieve this, the system is built using a modular architecture powered by AWS services such as S3, Glue, DynamoDB, Lambda, and Fargate. The scraper is designed to run automatically on every match day at specific intervals, ensuring timely updates of league tables. The project also emphasizes parallel scraping capabilities using AWS Fargate, enabling simultaneous data extraction from multiple leagues and multiple urls without resource conflicts or bottlenecks.

The project successfully delivered a scalable and reusable scraping pipeline capable of handling real-time football league standings across major European leagues. A dynamic

web dashboard has been implemented to display live league standings, highlight Champions League qualification spots, and track relegation zones. This solution lays the groundwork for future expansion into advanced sports analytics, additional leagues, and enhanced fan engagement tools.

III. Review of the Problem Being Addressed

In the modern sports analytics landscape, consistent and structured access to live and historical match statistics is crucial. However, football data is often spread across websites, making aggregation and daily analysis difficult. This project addresses the challenge of building a reliable data ingestion pipeline to extract and store structured information from publicly available sources.

The specific problem addressed is building an automated pipeline that scrapes English Premier League (EPL) standings and team statistics from the ESPN website and presents them in a format suitable for querying, analysis, and visualization. One major issue that arises in this domain is respecting scraping limitations imposed by websites via `robots.txt`, which may restrict automated access to parts of a website. Thus, identifying feasible scraping endpoints and formats is a foundational challenge.

IV. Technical Approach

Our technical approach includes the following components:

1) Scrapy
Scrapy is an open-source web scraping framework used to extract live football league standings. It was chosen for its speed and reliability and is used to collect data from ESPN for multiple leagues.

2) Docker & AWS ECR
Docker is a containerization tool used to package our Scrapy spiders, and AWS ECR (Elastic Container Registry) stores these containers. This ensures consistency across environments and simplifies deployment.

3) AWS Fargate
AWS Fargate is a serverless compute engine used to run containerized scraping jobs in parallel. It enables scalable, hands-off execution for multiple leagues simultaneously.

4) AWS EventBridge
EventBridge is an event scheduler used to trigger scraping tasks automatically each day at a fixed time. It ensures timely and consistent data updates without manual intervention.

5) AWS S3
Amazon S3 is a cloud storage service used to store the extracted standings data in .jsonl format. It offers scalable and durable storage for raw and processed data.

6) AWS Lambda
Lambda is a serverless compute service used to automate the ingestion of data from S3 into DynamoDB. It ensures smooth and event-driven data processing.

7) DynamoDB
DynamoDB is a NoSQL database used to store structured league standings for fast querying and integration into dashboards.

8) AWS Glue
Glue is a data integration service used to catalog, transform, and prepare data. It helps in structuring raw JSON data and making it queryable via Athena.

9) AWS Athena
Athena is an interactive query service used to analyze data stored in S3. It allows quick SQL-based queries on the processed league standings.

10) AWS QuickSight
QuickSight is a business intelligence tool used to visualize live standings and trends. It powers the final dashboard seen by end users.

11) AWS CloudWatch & SNS
CloudWatch monitors the health of scraping and data pipelines, while SNS sends real-time alerts for failures. Together, they ensure system reliability and transparency.

V. Architecture

This section outlines the system architecture and the technologies employed to extract, process, and present live football league standings data. This process is also automated for user access and interaction, ensuring that the latest data is consistently available through dashboards without manual intervention.

The architecture leverages a combination of powerful AWS services and containerization technologies to create a robust, scalable, and automated system for collecting, processing, and visualizing live football league standings. Scrapy powers the data extraction, while AWS services such as Fargate, Lambda, and Glue ensure that the data pipeline remains efficient and fault-tolerant. Through real-time data ingestion, transformation, and visualization, this system enables up-to-date insights into football league performance at scale.

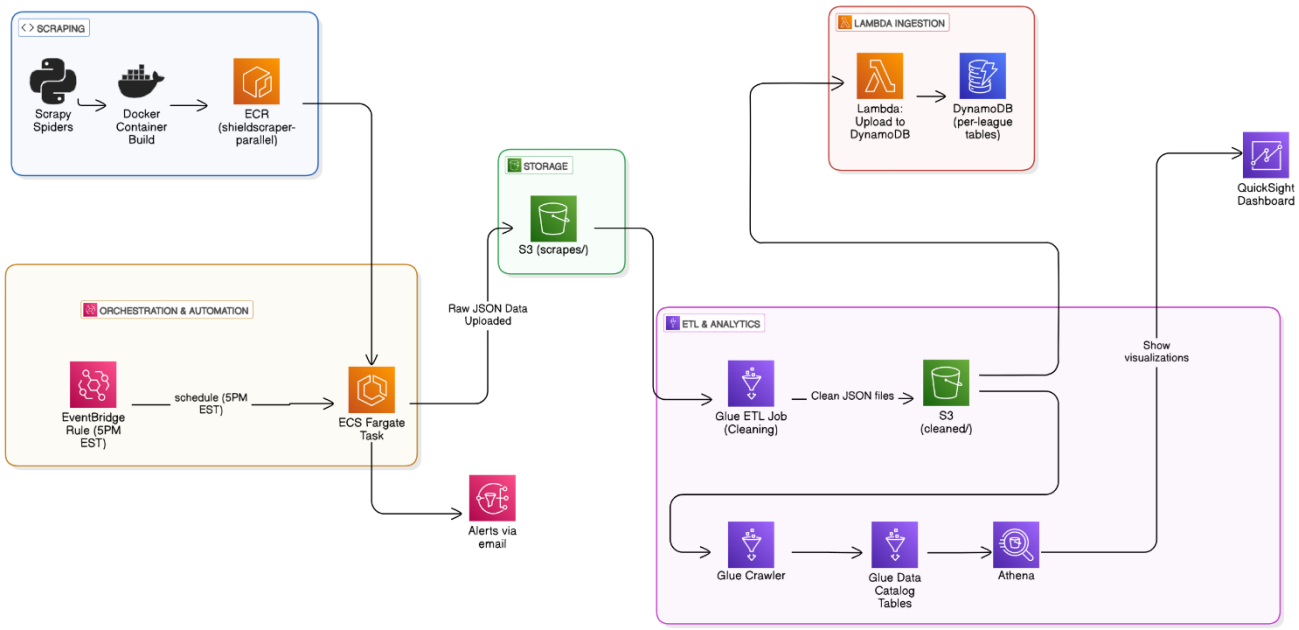


Figure.1 Architecture for SheildScrape

VI. Issues & Lessons Learned

A. Initial Issues:

- Initially, AWS Glue recognized the JSON output as a single array object rather than individual rows.
- The crawler failed to infer a row-based schema, breaking ETL operations.

B. Resolution:

- Switched from **.json** to **.jsonl** format, writing one JSON object per line.
- Re-ran the Glue Crawler to correctly infer schema with each JSON object representing a team

C. Athena Verification:

- Verified data correctness via Athena queries on the Glue table.
- Added logging in Glue jobs to verify row counts before writing to DynamoDB.

D. Feasibility:

- ESPN's standings page provided a clean, publicly available HTML table structure for scraping.
- Rate limits were not triggered, but we acknowledge potential access issues and are ready to switch sources if needed.

E. Automation Script:

- Encountered scheduling challenges as tasks were lined up sequentially after parallel computing. Multiple resources used required precise execution timing for the initial script to prevent null or outdated data ingestion.
- Addressed by explicitly configuring AWS EventBridge schedules to ensure that the scraping process started at a clearly defined and consistent timeframe, reducing the likelihood of data integrity issues.

F. Cost Considerations (Redshift vs. DynamoDB/Athena):

- Initially considered Amazon Redshift due to its ease of use and robust capabilities but decided against it due to higher operational costs.
- Opted to transfer data to DynamoDB for cost-effective storage and utilized Athena for maintaining the schema, balancing cost efficiency with performance needs.

G. Data Compatibility with Athena:

- Scraped data initially did not conform to Athena's query-compatible format, complicating direct visualization in QuickSight.
- Implemented AWS Glue scripts to clean and transform the data into a suitable format, ensuring seamless integration and effective visualization using QuickSight.

H. Lambda and Scrapy Integration:

- AWS Lambda does not natively support Scrapy scripts, which led to integration challenges.

  Resolved by containerizing Scrapy scripts with Docker and uploading them to Amazon ECR, facilitating the execution of scraping tasks through ECS Fargate, thereby maintaining seamless automation and execution reliability.

VII. Implementation

The architecture of ShieldScraper (Fig.1) consists of several key components designed for scalability, automation, and reliability. Each stage of the pipeline is integrated into a seamless flow, from web scraping to real-time data visualization. The entire process of ShieldScraper is fully automated through a series of well-integrated AWS services. The automation begins when AWS EventBridge triggers the scraping tasks at a set time, eliminating the need for manual initiation. This automated orchestration ensures that data is scraped, cleaned, and uploaded without human intervention.

A. Scraping and Containerization

The web scraping is powered by Scrapy, a robust and efficient Python-based web scraping framework. Scrapy spiders handle multiple concurrent scraping tasks, each designed to extract data from different sources. To ensure consistency in the runtime environment, all scraping tasks are containerized using Docker. This approach guarantees the same environment across various stages of deployment, ensuring smooth operation on different platforms, including AWS. The containerized spiders are deployed to AWS ECS Fargate, providing a serverless compute platform that scales automatically according to demand.

B. Orchestration and Automation

To manage the execution of scraping tasks, AWS EventBridge is used for orchestration. EventBridge schedules the scraping tasks to run daily at 5 PM EST, triggering ECS Fargate to execute the containerized spiders. The use of EventBridge ensures that the entire process is fully automated and runs without manual intervention. This system design guarantees high availability, consistent execution, and fault tolerance, allowing the pipeline to operate seamlessly on a scale.

C. Data Storage and Management

Once the data is scraped, it is stored in AWS S3 buckets. S3 provides secure, scalable, and durable storage, ensuring that the scraped data is safely stored and easily accessible for further processing. AWS Lambda functions are utilized to automatically ingest the data and structure it into DynamoDB tables. This step optimizes query performance and retrieval, enabling fast, reliable access to the data for further analysis.

D. ETL and Analytical Processing

To process the raw data and prepare it for analysis, AWS Glue is used to perform ETL operations. The raw data, stored in JSON format in S3, is transformed into structured datasets. AWS Glue Crawlers automatically catalog the metadata, allowing the data to be queried efficiently using AWS Athena. Athena enables serverless, interactive querying of the data without the need to manage infrastructure, providing fast insights for analysis.

E. Visualization and Real-time Monitoring

AWS QuickSight is integrated into the system to provide dynamic data visualization. QuickSight dashboards offer an intuitive interface for analyzing the processed data, enabling users to derive actionable insights. For real-time monitoring, AWS CloudWatch is used in conjunction with AWS SNS to monitor system health and performance. CloudWatch captures logs and metrics, while SNS sends immediate email alerts in the event of system issues or anomalies, ensuring timely identification and resolution.

Below attached in Fig.2 and Fig.3 are the snapshots of our dashboard. In Fig.2, the bar plot represents the number of wins registered by each team in their leagues and the top four teams from each league (highlighted by orange color) compete against each other at the UEFA Champions League. We have visualized and analyzed namely 3 leagues the English Premier League, Spanish League, and French League. Fig.3 represents the dashboard snapshot of various visualizations such as number of wins versus losses, a treemap visualization highlighting the points scored by each team suggesting the bigger the block, more points are scored by that team, goal efficiency compared to the total losses, and teams' performance based on draws and points scored.

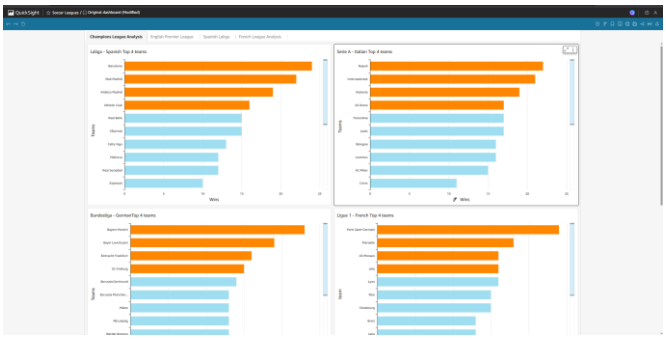


Figure.2 AWS QuickSight Dashboard – Champions League



Figure.3 AWS QuickSight Dashboard – Spanish League

VIII. Evaluation

The performance of ShieldScraper was evaluated across multiple dimensions to assess its scalability, reliability, and efficiency in a real-world scenario.

A. Execution Time and Latency

ECS Fargate demonstrated consistent and prompt execution of containerized spiders, with negligible latency. The scraping tasks were completed efficiently, ensuring timely data extraction even at scale.

B. ETL Performance

AWS Glue was able to successfully transform datasets up to 100MB within two minutes, showcasing its capability to

handle large datasets with minimal processing time. This indicates the robustness of the ETL process and the scalability of the solution.

C. Data Query Performance

DynamoDB exhibited efficient handling of increased query loads, with no noticeable performance degradation even when querying large volumes of structured data. The integration with Lambda and DynamoDB allowed for rapid, scalable data access.

D. Real-Time Analytics and Monitoring

AWS QuickSight provided real-time analytics, with interactive dashboards allowing for dynamic data exploration. CloudWatch, integrated with SNS, facilitated real-time monitoring, ensuring that any system issues were immediately identified and addressed.

IX. Discussion

ShieldScraper effectively addresses several critical aspects, such as automation, scalability, and reliability, demonstrating substantial improvement over manual data collection methods. The fully automated pipeline leveraging AWS ECS Fargate and EventBridge significantly reduces manual intervention, minimizing human error and ensuring consistent data updates. Real-time monitoring integrated with AWS CloudWatch and SNS ensures operational transparency, promptly notifying stakeholders of system anomalies, thereby increasing system reliability and user trust.

X. Limitations

Despite these strengths, the current scraping approach heavily depends on the structural consistency of targeted web pages, such as those from ESPN. Any unanticipated changes in website structure could disrupt the scraping process, resulting in incomplete or erroneous data capture. Additionally, the scraper currently does not incorporate adaptive methodologies to dynamically detect and adjust to webpage structural changes, potentially requiring manual intervention to update scraping scripts whenever changes occur. Further, compliance with website terms of service and scraping limitations imposed by the site administrators (such as rate limiting or IP blocking) pose additional operational risks that need continuous monitoring.

XI. Future Work

Future enhancements for ShieldScraper include implementing machine learning algorithms to create adaptive scraping capabilities, allowing the pipeline to autonomously detect and adjust to structural changes on target websites. This would significantly reduce maintenance overhead and enhance the system's robustness. Another important area for future work involves the integration of more sophisticated anomaly detection and predictive analytics to proactively identify potential issues before they occur, further minimizing downtime and improving overall pipeline resilience. Lastly, expanding the scope of ShieldScraper to include additional data sources and more leagues globally would enhance the solution's applicability, offering richer insights and catering to a broader audience base.

XII. Conclusion

ShieldScraper presents a comprehensive, cloud-based solution for web scraping, data processing, and real-time analytics. By utilizing AWS services such as ECS Fargate, Lambda, DynamoDB, Glue, Athena, QuickSight, and CloudWatch, the system achieves high scalability, reliability, and efficiency. The performance evaluation demonstrates the pipeline's ability to process and query large datasets in real time, ensuring that the system can handle increasing loads without compromising on performance. ShieldScraper's architecture and design offer valuable insights for building scalable and automated data pipelines for modern web scraping applications.

XIII. Roles of Team Members

1) Mate, Shivali – Developed the job scrapers and implemented containerization using Docker for scalable deployments.
2) Sonawane, Varunkumar – Designed and built the ETL pipeline integrating AWS services such as Glue, S3, DynamoDB, and Athena.
3) Dhuru, Aryan Mayur - Implemented end-to-end automation, including job scheduling, orchestration with EventBridge, and scripting of cloud workflows.
4) Ambol, Suhana - Created impactful data visualizations using Amazon Quicksight and led project documentation efforts.

XIV. References

[1] Scrapy Documentation: https://docs.scrapy.org/
[2] S3 Documentation - https://docs.aws.amazon.com/s3/
[3] Amazon Web Services (AWS) Documentation: https://aws.amazon.com/documentation/
[4] AWS Glue Documentation - https://docs.aws.amazon.com/glue/
[5] Amazon DynamoDB Documentation - https://docs.aws.amazon.com/dynamodb/
[6] AWS Identity and Access Management Documentation - https://docs.aws.amazon.com/iam/
[7] AWS Security Best Practices: https://aws.amazon.com/whitepapers/aws-security-best-practices/
[8] AWS QuickSight Documentation: https://docs.aws.amazon.com/quicksight/latest/user/welcome.html